

1. Judul Praktikum

Show command palette (Ctrl+Shift+P) - Modul 3 (Tugas Praktikum & Diskusi)

2. Tujuan Praktikum

Mampu mengimplementasikan logika struktur data dasar, operasi ADT (Stack dan Queue), serta melakukan perhitungan perbandingan iterasi untuk membuktikan teori kompleksitas algoritma Big-O Notation.

3. Dasar Teori

Tugas ini menerapkan teori struktur data List dan pembuatan sistem *Stack* (LIFO - Last In First Out) serta *Queue* (FIFO - First In First Out). Untuk perbandingan algoritma, perhitungan *counter* diletakkan di dalam perulangan tunggal (Linear $O(n)$) dan perulangan bersarang (Kuadratik $O(n^2)$) untuk memberikan bukti konkrit seberapa banyak proses memori yang dibutuhkan.

4. Langkah Praktikum

Berikut adalah penyelesaian kode terstruktur mulai dari Tugas 1 hingga Tugas 8, dengan urutan dari Level Mudah hingga Sulit. (Tugas 9 berbentuk analisis teks dan disertakan di bagian Analisis).

```
1 print("===== LEVEL MUDAH =====")
2
3 print("--- TUGAS 1 (List Dasar) ---")
4 angka_list = [15, 25, 35, 45, 55]
5 print(f"Seluruh data list: {angka_list}\n")
6
7
8
9 print("===== LEVEL SEDANG =====")
10
11 print("--- TUGAS 4 (Menghitung Jumlah Iterasi Loop) ---")
12 counter_loop = 0
13 for i in range(10):
14     counter_loop += 1
15 print(f"Jumlah iterasi pada single loop n=10 adalah: {counter_loop}\n")
16
17
18
19 print("===== LEVEL SULIT =====")
20
21 print("--- TUGAS 7 (Bandingkan Efisiensi 2 Algoritma Pencarian) ---")
22 data_terurut = [1, 3, 5, 7, 9, 11, 13, 15]
23 target_cari = 13
24
25 # 1. Linear Search
26 iterasi_linear = 0
27 for val in data_terurut:
28     iterasi_linear += 1
29     if val == target_cari:
30         break
31 print(f"Linear Search  $O(n)$  butuh: {iterasi_linear} iterasi")
32
33 # 2. Binary Search (Implementasi sederhana)
34 iterasi_binary = 0
35 kiri, kanan = 0, len(data_terurut) - 1
36 while kiri <= kanan:
37     iterasi_binary += 1
38     tengah = (kiri + kanan) // 2
39     if data_terurut[tengah] == target_cari:
40         break
41     elif data_terurut[tengah] < target_cari:
42         kiri = tengah + 1
43     else:
44         kanan = tengah - 1
45 print(f"Binary Search  $O(\log n)$  butuh: {iterasi_binary} iterasi")
46 print("Kesimpulan: Binary search jauh lebih efisien untuk data terurut.\n")
```

```
===== LEVEL MUDAH =====
--- TUGAS 1 (List Dasar) ---
Seluruh data list: [15, 25, 35, 45, 55]
```

```
===== LEVEL SEDANG =====
--- TUGAS 4 (Menghitung Jumlah Iterasi Loop) ---
Jumlah iterasi pada single loop n=10 adalah: 10
```

```
===== LEVEL SULIT =====
--- TUGAS 7 (Bandingkan Efisiensi 2 Algoritma Pencarian) ---
Linear Search  $O(n)$  butuh: 7 iterasi
```

```
Binary Search O(log n) butuh: 3 iterasi  
Kesimpulan: Binary search jauh lebih efisien untuk data terurut.
```

Show command palette (Ctrl+Shift+P)

- **Tugas 4 (Single Loop):** Kompleksitasnya adalah linear, $O(n)$. Jika loop dijalankan 10 kali, jumlah eksekusinya tepat 10 kali.
- **Tugas 7 (Pencarian):** Terbukti bahwa Binary Search (logaritmik $O(\log n)$) jauh lebih efisien daripada Linear Search (linear $O(n)$) karena memotong setengah ruang pencarian di setiap iterasi.

6. Kesimpulan & Jawaban Diskusi

Jawaban Diskusi:

1. **Mengapa struktur data penting dalam pemrograman?** Karena struktur data menentukan bagaimana data diorganisasikan, yang secara langsung mempengaruhi efisiensi penggunaan memori, kecepatan pemrosesan algoritma, dan kemudahan dalam pengembangan kode.
2. **Apa perbedaan struktur data linear dan non-linear?** Struktur data linear mengatur datanya secara berurutan atau sekuensial (seperti Array dan List). Sedangkan struktur data non-linear tidak disusun berurutan, melainkan membentuk hierarki atau jaringan percabangan (seperti Tree dan Graph).
3. **Mengapa algoritma harus efisien?** Agar program yang dibuat dapat memproses operasi dengan lebih cepat, menggunakan kapasitas memori yang lebih sedikit/optimal, serta mampu bertahan dan tidak *crash* ketika harus menangani jumlah data (dataset) berskala sangat besar.
4. **Apa fungsi Big-O Notation?** Berfungsi sebagai alat ukur matematika abstrak untuk menggambarkan dan menganalisis seberapa signifikan waktu eksekusi suatu algoritma akan bertambah (bertumbuh) seiring dengan bertambahnya jumlah ukuran data input (n).
5. **Mengapa nested loop lebih lambat dibanding single loop?** Karena *nested loop* memiliki kompleksitas Kuadratik $O(n^2)$. Artinya, perulangan di dalam perulangan mengeksekusi instruksi berulang kali secara eksponensial (misal $n=100$, operasinya 10000 kali), tidak linier seperti *single loop* yang mengeksekusi sebanding dengan jumlah datanya saja (100 kali).